

Programa2.Estandarizacion

September 24, 2026

Programa2.Estandarizacion

September 23, 2026

Link a mi GitHub

<https://github.com/kevinsuas433/Machine2026.git>

```
[1]: # =====  
# Importación de librerías  
# =====  
  
# Importamos las herramientas de preprocesamiento de  
# Scikit-Learn que utilizaremos para estandarizar los datos.  
from sklearn import preprocessing  
  
# Importamos NumPy para trabajar con matrices y arreglos numéricos.  
import numpy as np
```

```
[3]: # =====  
# Carga de datos  
# =====  
  
# Creamos la matriz de datos de entrenamiento.  
# Cada fila representa un registro y cada columna  
# representa una característica.  
X_train = np.array([  
    [1., -1., 2.],  
    [2., 0., 0.],  
    [0., 1., -1.]  
)  
  
# Mostramos los datos cargados.  
print(X_train)
```

```
[[ 1. -1.  2.]  
 [ 2.  0.  0.]  
 [ 0.  1. -1.]]
```

```
[4]: # =====
# Limpieza de datos: estandarización
# =====

# Creamos el objeto StandardScaler encargado de
# estandarizar las características.
standardizer = preprocessing.StandardScaler()

# Ajustamos el estandarizador a los datos de entrenamiento
# y aplicamos la transformación.
X_train_std = standardizer.fit_transform(X_train)

# Mostramos los datos después de la estandarización.
print(X_train_std)
```

```
[[ 0.          -1.22474487  1.33630621]
 [ 1.22474487  0.          -0.26726124]
 [-1.22474487  1.22474487 -1.06904497]]
```

```
[5]: # =====
# Creación de una segunda matriz de datos
# =====

# Creamos una segunda matriz con los mismos datos de entrenamiento.
# Esta matriz será utilizada para comprobar que los métodos
# fit() y transform() por separado producen el mismo resultado.
X_train_2 = np.array([
    [1., -1., 2.],
    [2., 0., 0.],
    [0., 1., -1.]
])
```

```
[6]: # =====
# Aplicación de fit() y transform() por separado
# =====

# Creamos un nuevo objeto StandardScaler.
f = preprocessing.StandardScaler()

# Ajustamos el estandarizador utilizando los datos de entrenamiento.
# El método fit() calcula los parámetros necesarios para
# realizar posteriormente la estandarización.
f = f.fit(X_train)

# Aplicamos la transformación sobre la segunda matriz.
# Aquí utilizamos únicamente transform(), ya que el objeto
# ya fue ajustado previamente con fit().
X_train_2_std = f.transform(X_train_2)
```

```
# Mostramos los datos después de la estandarización.  
print(X_train_2_std)
```

```
[[ 0.          -1.22474487  1.33630621]  
 [ 1.22474487  0.          -0.26726124]  
 [-1.22474487  1.22474487 -1.06904497]]
```

```
[7]: # =====  
# Comprobación de las matrices estandarizadas  
# =====  
  
# Comparamos la matriz obtenida mediante fit_transform()  
# con la matriz obtenida utilizando fit() y transform()  
# por separado.  
resultado = np.array_equiv(  
    X_train_std,  
    X_train_2_std  
)  
  
# Mostramos el resultado de la comparación.  
print(resultado)
```

True

```
[ ]:
```